

Competitive Programming

Section 68 Editorial: Advanced Tree Decompositions

Competitive Programming Course

Table of contents

1	A. Path Affine Composition	3
2	B. Toggle Nearest Beacon	8
3	C. Marked Pair Distances	12
4	D. Subtree Mode Sum	16
5	Stress testing and carry-forward	19

1 A. Path Affine Composition

Store every HLD interval in a segment tree with forward and backward affine composition. Merging adjacent intervals composes left-forward then right-forward; backward uses right-backward then left-backward. Split a path into upward u chunks, a same-chain middle, and downward v chunks, preserving their order.

Correctness. Segment induction proves both stored directions. Each HLD chunk is a contiguous part of the path. The accumulation rules concatenate these disjoint chunks in exact $u \rightarrow v$ order, so the final affine function is precisely the sequence requested. Applying it to x gives the answer.

Preprocessing is $O(n)$. Updates cost $O(\log n)$ and queries $O(\log^2 n)$; memory is $O(n)$. Test reversed paths with functions that do not commute.

```
#include <bits/stdc++.h>
using namespace std;
constexpr long long MOD = 998244353;
struct Function {
    long long a = 1, b = 0;
};
Function then(Function first, Function second) {
    return {second.a * first.a % MOD, (second.a * first.b + second.b) % MOD};
}
struct Aggregate {
    Function forward, backward;
};
Aggregate merge(Aggregate left, Aggregate right) {
    return {then(left.forward, right.forward), then(right.backward, left.backward)};
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<Function> fn(n);
    for (auto& f : fn) cin >> f.a >> f.b;
    vector<vector<int>> g(n);
    for (int i = 1, u, v; i < n; i++) {
        cin >> u >> v;
        --u;
        --v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> parent(n, -1), depth(n), order{0};
```

```

for (int i = 0; i < (int)order.size(); i++) {
    int u = order[i];
    for (int v : g[u])
        if (v != parent[u]) parent[v] = u, depth[v] = depth[u] + 1, order.push_back(v);
}
vector<int> size(n, 1), heavy(n, -1);
for (int i = n - 1; i > 0; i--) {
    int u = order[i], p = parent[u];
    size[p] += size[u];
    if (heavy[p] < 0 || size[u] > size[heavy[p]]) heavy[p] = u;
}
vector<int> head(n), position(n);
int timer = 0;
vector<pair<int, int>> chains{{0, 0}};
while (!chains.empty()) {
    auto [start, h] = chains.back();
    chains.pop_back();
    for (int u = start; u != -1; u = heavy[u]) {
        head[u] = h;
        position[u] = timer++;
        for (int v : g[u])
            if (parent[v] == u && v != heavy[u]) chains.push_back({v, v});
    }
}
int base = 1;
while (base < n) base *= 2;
vector<Aggregate> tree(2 * base);
for (int u = 0; u < n; u++) tree[base + position[u]] = {fn[u], fn[u]};
for (int i = base - 1; i; i--) tree[i] = merge(tree[2 * i], tree[2 * i + 1]);
auto range = [&](int l, int r) {
    Aggregate left{}, right{};
    for (l += base, r += base + 1; l < r; l /= 2, r /= 2) {
        if (l & 1) left = merge(left, tree[l++]);
        if (r & 1) right = merge(tree[--r], right);
    }
    return merge(left, right);
};
while (q--) {
    char type;
    cin >> type;
    if (type == 'U') {
        int u;
        Function f;
        cin >> u >> f.a >> f.b;
        --u;
        int p = base + position[u];
        tree[p] = {f, f};
        for (p /= 2; p; p /= 2) tree[p] = merge(tree[2 * p], tree[2 * p + 1]);
    } else {
        int u, v;
        long long x;
    }
}

```

```

cin >> u >> v >> x;
--u;
--v;
Function left{}, right{};
while (head[u] != head[v]) {
    if (depth[head[u]] >= depth[head[v]]) {
        left = then(left, range(position[head[u]], position[u]).backward);
        u = parent[head[u]];
    } else {
        right = then(range(position[head[v]], position[v]).forward, right);
        v = parent[head[v]];
    }
}
Function middle = position[u] <= position[v]
                    ? range(position[u], position[v]).forward
                    : range(position[v], position[u]).backward;
Function all = then(then(left, middle), right);
cout << (all.a * x + all.b) % MOD << '\n';
}
}
}

```

```

import sys

MOD = 998_244_353
IDENTITY = (1, 0)

def compose(first, second):
    return second[0] * first[0] % MOD, (second[0] * first[1] + second[1]) % MOD

def merge(left, right):
    return compose(left[0], right[0]), compose(right[1], left[1])

def main():
    tokens = sys.stdin.buffer.read().split()
    if not tokens:
        return
    idx = 0
    n = int(tokens[idx])
    q = int(tokens[idx + 1])
    idx += 2
    functions = []
    for _ in range(n):
        functions.append((int(tokens[idx]), int(tokens[idx + 1])))
        idx += 2
    graph = [[] for _ in range(n)]
    for _ in range(n - 1):
        u = int(tokens[idx]) - 1

```

```

        v = int(tokens[idx + 1]) - 1
        idx += 2
        graph[u].append(v)
        graph[v].append(u)
parent = [-1] * n
depth = [0] * n
order = [0]
for u in order:
    for v in graph[u]:
        if v != parent[u]:
            parent[v] = u
            depth[v] = depth[u] + 1
            order.append(v)
size = [1] * n
heavy = [-1] * n
for u in reversed(order[1:]):
    p = parent[u]
    size[p] += size[u]
    if heavy[p] < 0 or size[u] > size[heavy[p]]:
        heavy[p] = u
head = [0] * n
position = [0] * n
timer = 0
chains = [(0, 0)]
while chains:
    start, h = chains.pop()
    u = start
    while u != -1:
        head[u] = h
        position[u] = timer
        timer += 1
        for v in graph[u]:
            if parent[v] == u and v != heavy[u]:
                chains.append((v, v))
        u = heavy[u]
base = 1
while base < n:
    base *= 2
identity = (IDENTITY, IDENTITY)
tree = [identity for _ in range(2 * base)]
for u, f in enumerate(functions):
    tree[base + position[u]] = (f, f)
for p in range(base - 1, 0, -1):
    tree[p] = merge(tree[2 * p], tree[2 * p + 1])

def range_query(left, right):
    a = b = identity
    left += base
    right += base + 1
    while left < right:
        if left & 1:

```

```

        a = merge(a, tree[left])
        left += 1
    if right & 1:
        right -= 1
        b = merge(tree[right], b)
    left //= 2
    right //= 2
    return merge(a, b)

out = []
for _ in range(q):
    kind = tokens[idx].decode()
    idx += 1
    if kind == "U":
        u = int(tokens[idx]) - 1
        f = (int(tokens[idx + 1]), int(tokens[idx + 2]))
        idx += 3
        p = base + position[u]
        tree[p] = (f, f)
        p //= 2
        while p:
            tree[p] = merge(tree[2 * p], tree[2 * p + 1])
            p //= 2
    else:
        u = int(tokens[idx]) - 1
        v = int(tokens[idx + 1]) - 1
        x = int(tokens[idx + 2])
        idx += 3
        left = right = IDENTITY
        while head[u] != head[v]:
            if depth[head[u]] >= depth[head[v]]:
                left = compose(left, range_query(position[head[u]], position[u])[1])
                u = parent[head[u]]
            else:
                right = compose(
                    range_query(position[head[v]], position[v])[0], right
                )
                v = parent[head[v]]
        middle = (
            range_query(position[u], position[v])[0]
            if position[u] <= position[v]
            else range_query(position[v], position[u])[1]
        )
        whole = compose(compose(left, middle), right)
        out.append(str((whole[0] * x + whole[1]) % MOD))
print("\n".join(out))

if __name__ == "__main__":
    main()

```

2 B. Toggle Nearest Beacon

Build centroid decomposition and record every vertex's distance to each centroid ancestor. Each centroid owns a multiset of distances contributed by active vertices. Toggle inserts or erases one distance along the vertex's centroid path. Query minimizes its own distance to c plus the multiset minimum at c .

Correctness. For any query vertex u and active vertex v , their centroid paths share the first separator at which their original-tree path crosses between decomposition components; at that centroid c , $\text{dist}(u,v)=\text{dist}(u,c)+\text{dist}(v,c)$. The candidate is therefore present. Every candidate is the length of a concrete route through its centroid and cannot be smaller than the true distance. Taking the minimum is exact.

Build time is $O(n \log n)$. Toggle and query cost $O(\log^2 n)$ with multisets; memory is $O(n \log n)$. Python uses paired lazy-deletion heaps.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<vector<int>> g(n);
    for (int i = 1, u, v; i < n; i++) {
        cin >> u >> v;
        --u;
        --v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<char> blocked(n), active(n);
    vector<int> parent(n, -1), subtree(n);
    vector<vector<pair<int, int>>> path(n);
    function<void(int, int)> decompose = [&](int entry, int centroid_parent) {
        vector<int> nodes, par(n, -2);
        nodes.push_back(entry);
        par[entry] = -1;
        for (int i = 0; i < (int)nodes.size(); i++) {
            int u = nodes[i];
            for (int v : g[u])
                if (!blocked[v] && par[v] == -2) par[v] = u, nodes.push_back(v);
        }
        for (int i = nodes.size() - 1; i >= 0; i--) {
            int u = nodes[i];
            subtree[u] = 1;
        }
    };
    // ... (rest of the code)
}
```

```

        for (int v : g[u])
            if (!blocked[v] && par[v] == u) subtree[u] += subtree[v];
    }
    int total = nodes.size(), centroid = entry;
    for (int u : nodes) {
        int largest = total - subtree[u];
        for (int v : g[u])
            if (!blocked[v] && par[v] == u) largest = max(largest, subtree[v]);
        if (largest * 2 <= total) {
            centroid = u;
            break;
        }
    }
    parent[centroid] = centroid_parent;
    vector<tuple<int, int, int>> stack{{centroid, -1, 0}};
    while (!stack.empty()) {
        auto [u, p, d] = stack.back();
        stack.pop_back();
        path[u].push_back({centroid, d});
        for (int v : g[u])
            if (v != p && !blocked[v]) stack.push_back({v, u, d + 1});
    }
    blocked[centroid] = 1;
    for (int v : g[centroid])
        if (!blocked[v]) decompose(v, centroid);
};
decompose(0, -1);
vector<multiset<int>> distances(n);
while (q--) {
    char type;
    int u;
    cin >> type >> u;
    --u;
    if (type == 'T') {
        active[u] ^= 1;
        for (auto [c, d] : path[u]) {
            if (active[u])
                distances[c].insert(d);
            else
                distances[c].erase(distances[c].find(d));
        }
    } else {
        int answer = INT_MAX;
        for (auto [c, d] : path[u])
            if (!distances[c].empty()) answer = min(answer, d + *distances[c].begin());
        cout << (answer == INT_MAX ? -1 : answer) << '\n';
    }
}
}

```

```

import heapq, sys

def main():
    tokens = sys.stdin.buffer.read().split()
    if not tokens:
        return
    idx = 0
    n = int(tokens[idx])
    q = int(tokens[idx + 1])
    idx += 2
    graph = [[] for _ in range(n)]
    for _ in range(n - 1):
        u = int(tokens[idx]) - 1
        v = int(tokens[idx + 1]) - 1
        idx += 2
        graph[u].append(v)
        graph[v].append(u)
    blocked = [False] * n
    paths = [[] for _ in range(n)]
    subtree = [0] * n
    todo = [0]
    while todo:
        entry = todo.pop()
        nodes = [entry]
        parent = {entry: -1}
        for u in nodes:
            for v in graph[u]:
                if not blocked[v] and v not in parent:
                    parent[v] = u
                    nodes.append(v)
        for u in reversed(nodes):
            subtree[u] = 1 + sum(
                subtree[v]
                for v in graph[u]
                if not blocked[v] and parent.get(v) != None and parent.get(v) == u
            )
        total = len(nodes)
        centroid = entry
        for u in nodes:
            largest = total - subtree[u]
            for v in graph[u]:
                if not blocked[v] and parent.get(v) == u:
                    largest = max(largest, subtree[v])
            if largest * 2 <= total:
                centroid = u
                break
        stack = [(centroid, -1, 0)]
        while stack:
            u, p, d = stack.pop()
            paths[u].append((centroid, d))

```

```

        for v in graph[u]:
            if v != p and not blocked[v]:
                stack.append((v, u, d + 1))
        blocked[centroid] = True
        for v in graph[centroid]:
            if not blocked[v]:
                todo.append(v)
    added = [[] for _ in range(n)]
    removed = [[] for _ in range(n)]
    active = [False] * n

    def clean(c):
        while added[c] and removed[c] and added[c][0] == removed[c][0]:
            heapq.heappop(added[c])
            heapq.heappop(removed[c])

    out = []
    for _ in range(q):
        kind = tokens[idx].decode()
        u = int(tokens[idx + 1]) - 1
        idx += 2
        if kind == "T":
            active[u] = not active[u]
            for c, d in paths[u]:
                heapq.heappush(added[c] if active[u] else removed[c], d)
        else:
            answer = 10**18
            for c, d in paths[u]:
                clean(c)
                answer = min(answer, d + added[c][0]) if added[c] else answer
            out.append(str(-1 if answer == 10**18 else answer))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

3 C. Marked Pair Distances

Sort marked vertices by Euler time, add LCAs of consecutive entries, sort and deduplicate, then connect them by an ancestor stack. Initialize each marked virtual vertex with count one. Process virtual vertices bottom-up. For edge $p-u$, add

```
(dist[u]-dist[p]) * count[u] * (k-count[u]).
```

and transfer `count[u]` to `p`.

Correctness. The virtual tree preserves every branching LCA and compresses only degree-two unmarked paths, so its edge length equals the removed original path. Removing edge $p-u$ separates `count[u]` marks from the other `k-count[u]`; exactly their cross pairs use every original edge represented by it. Summing over virtual edges counts each pair once per edge on its path, which is exactly its distance.

Preprocessing costs $O(n \log n)$. A query costs $O(k \log k)$ and uses $O(k)$ temporary memory. Test one mark, ancestor relations, and LCAs absent from the input set.

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<vector<pair<int, int>>> g(n);
    for (int i = 1, u, v, w; i < n; i++) {
        cin >> u >> v >> w;
        --u;
        --v;
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    int log = 1;
    while ((1 << log) <= n) log++;
    vector<vector<int>> up(log, vector<int>(n));
    vector<int> tin(n), tout(n), depth(n);
    vector<int64> distance(n);
    int timer = 0;
    vector<tuple<int, int, int, int>> stack{{0, 0, 0, 0}};
    while (!stack.empty()) {
        auto [u, p, w, state] = stack.back();
        stack.pop_back();
        if (!state) {
```

```

    tin[u] = timer++;
    up[0][u] = p;
    for (int j = 1; j < log; j++) up[j][u] = up[j - 1][up[j - 1][u]];
    stack.push_back({u, p, w, 1});
    for (auto it = g[u].rbegin(); it != g[u].rend(); ++it)
        if (it->first != p) {
            depth[it->first] = depth[u] + 1;
            distance[it->first] = distance[u] + it->second;
            stack.push_back({it->first, u, it->second, 0});
        }
    } else
        tout[u] = timer - 1;
}

auto ancestor = [&](int u, int v) { return tin[u] <= tin[v] && tout[v] <= tout[u]; };
auto lca = [&](int u, int v) {
    if (ancestor(u, v)) return u;
    if (ancestor(v, u)) return v;
    for (int j = log - 1; j >= 0; j--)
        if (!ancestor(up[j][u], v)) u = up[j][u];
    return up[0][u];
};

vector<char> marked(n);
vector<int> count(n), virtual_parent(n);
while (q--) {
    int k;
    cin >> k;
    vector<int> nodes(k);
    for (int& u : nodes) {
        cin >> u;
        --u;
        marked[u] = 1;
    }
    sort(nodes.begin(), nodes.end(), [&](int a, int b) { return tin[a] < tin[b]; });
    int original = nodes.size();
    for (int i = 1; i < original; i++) nodes.push_back(lca(nodes[i - 1], nodes[i]));
    sort(nodes.begin(), nodes.end(), [&](int a, int b) { return tin[a] < tin[b]; });
    nodes.erase(unique(nodes.begin(), nodes.end()), nodes.end());
    vector<int> st;
    for (int u : nodes) {
        while (!st.empty() && !ancestor(st.back(), u)) st.pop_back();
        virtual_parent[u] = st.empty() ? -1 : st.back();
        st.push_back(u);
        count[u] = marked[u];
    }
    int64 answer = 0;
    for (int i = nodes.size() - 1; i >= 0; i--) {
        int u = nodes[i];
        int p = virtual_parent[u];
        if (p != -1) {
            answer += (distance[u] - distance[p]) * count[u] * (k - count[u]);
            count[p] += count[u];
        }
    }
}

```

```

    }
}
cout << answer << '\n';
for (int u : nodes) marked[u] = 0;
}
}

```

```
import sys
```

```

def main():
    tokens = sys.stdin.buffer.read().split()
    if not tokens:
        return
    idx = 0
    n = int(tokens[idx])
    q = int(tokens[idx + 1])
    idx += 2
    graph = [[] for _ in range(n)]
    for _ in range(n - 1):
        u = int(tokens[idx]) - 1
        v = int(tokens[idx + 1]) - 1
        w = int(tokens[idx + 2])
        idx += 3
        graph[u].append((v, w))
        graph[v].append((u, w))
    log = n.bit_length()
    up = [[0] * n for _ in range(log)]
    tin = [0] * n
    tout = [0] * n
    distance = [0] * n
    timer = 0
    stack = [(0, 0, 0)]
    while stack:
        u, p, state = stack.pop()
        if state == 0:
            tin[u] = timer
            timer += 1
            up[0][u] = p
            for j in range(1, log):
                up[j][u] = up[j - 1][up[j - 1][u]]
            stack.append((u, p, 1))
            for v, w in reversed(graph[u]):
                if v != p:
                    distance[v] = distance[u] + w
                    stack.append((v, u, 0))
        else:
            tout[u] = timer - 1

    def ancestor(u, v):
        return tin[u] <= tin[v] <= tout[u]

```

```

def lca(u, v):
    if ancestor(u, v):
        return u
    if ancestor(v, u):
        return v
    for j in range(log - 1, -1, -1):
        if not ancestor(up[j][u], v):
            u = up[j][u]
    return up[0][u]

marked = [False] * n
count = [0] * n
parent = [-1] * n
out = []
for _ in range(q):
    k = int(tokens[idx])
    idx += 1
    nodes = [int(x) - 1 for x in tokens[idx : idx + k]]
    idx += k
    for u in nodes:
        marked[u] = True
    nodes.sort(key=tin.__getitem__)
    nodes += [lca(nodes[i - 1], nodes[i]) for i in range(1, k)]
    nodes = sorted(set(nodes), key=tin.__getitem__)
    st = []
    for u in nodes:
        while st and not ancestor(st[-1], u):
            st.pop()
        parent[u] = st[-1] if st else -1
        st.append(u)
        count[u] = int(marked[u])
    answer = 0
    for u in reversed(nodes):
        p = parent[u]
        if p != -1:
            answer += (distance[u] - distance[p]) * count[u] * (k - count[u])
            count[p] += count[u]
    out.append(str(answer))
    for u in nodes:
        marked[u] = False
print("\n".join(out))

if __name__ == "__main__":
    main()

```

4 D. Subtree Mode Sum

Process vertices bottom-up. Reuse the largest child frequency map, merge every smaller child map into it, then add the current color. Maintain the maximum frequency and sum of colors attaining it whenever a count increases.

Correctness. By induction, child maps contain exactly their disjoint subtrees. Their union plus the current vertex is exactly the current subtree, independent of merge order. The maintained maximum resets its sum when a strictly larger count appears and adds a color when it ties, so the reported sum is exact.

Every color entry moves only into a map at least twice as large, at most $O(\log n)$ times. Expected time is $O(n \log n)$ and memory $O(n)$. Test ties, all equal colors, all distinct colors, chains, and stars.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<int> color(n);
    for (int& x : color) cin >> x;
    vector<vector<int>> g(n);
    for (int i = 1, u, v; i < n; i++) {
        cin >> u >> v;
        --u;
        --v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> parent(n, -1), order{0};
    for (int i = 0; i < (int)order.size(); ++i) {
        int u = order[i];
        for (int v : g[u])
            if (v != parent[u]) parent[v] = u, order.push_back(v);
    }
    vector<unordered_map<int, int>*> maps(n);
    vector<int> maximum(n);
    vector<long long> answer(n);
    for (int at = n - 1; at >= 0; --at) {
        int u = order[at], base = -1;
        for (int v : g[u])
            if (parent[v] == u && (base < 0 || maps[v]->size() > maps[base]->size()))
                base = v;
    }
```

```

if (base < 0)
    maps[u] = new unordered_map<int, int>();
else
    maps[u] = maps[base], maximum[u] = maximum[base], answer[u] = answer[base];
auto add = [&](int c, int amount) {
    int value = ((*maps[u])[c] += amount);
    if (value > maximum[u])
        maximum[u] = value, answer[u] = c;
    else if (value == maximum[u])
        answer[u] += c;
};
for (int v : g[u])
    if (parent[v] == u && v != base) {
        for (auto [c, amount] : *maps[v]) add(c, amount);
        delete maps[v];
    }
add(color[u], 1);
}
for (long long value : answer) cout << value << ' ';
cout << '\n';
delete maps[0];
}

```

```

import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    colors = data[1 : n + 1]
    idx = n + 1
    graph = [[] for _ in range(n)]
    for _ in range(n - 1):
        u = data[idx] - 1
        v = data[idx + 1] - 1
        idx += 2
        graph[u].append(v)
        graph[v].append(u)
    parent = [-1] * n
    order = [0]
    for u in order:
        for v in graph[u]:
            if v != parent[u]:
                parent[v] = u
                order.append(v)
    maps = [None] * n
    maximum = [0] * n
    answer = [0] * n
    for u in reversed(order):

```

```

children = [v for v in graph[u] if parent[v] == u]
base = max(children, key=lambda v: len(maps[v]), default=-1)
if base < 0:
    current = {}
    best = total = 0
else:
    current = maps[base]
    best = maximum[base]
    total = answer[base]

def add(color, amount):
    nonlocal best, total
    value = current.get(color, 0) + amount
    current[color] = value
    if value > best:
        best = value
        total = color
    elif value == best:
        total += color

for v in children:
    if v != base:
        for color, amount in maps[v].items():
            add(color, amount)
        maps[v].clear()
add(colors[u], 1)
maps[u] = current
maximum[u] = best
answer[u] = total
print(*answer)

if __name__ == "__main__":
    main()

```

5 Stress testing and carry-forward

The generators use explicit path reconstruction, all-pairs tree distances, and complete subtree color counters. The recurring discovery move is to choose the small boundary: logarithmically many chains/centroids, $O(k)$ virtual vertices, or entries that can move only logarithmically often.